

**UNIVERSIDADE FEDERAL DE ALAGOAS
INSTITUTO DE COMPUTAÇÃO
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO**



DAVI SILVA DE MELO LINS

Relatório Milestone 3 - Projeto: Rede de Relacionamentos Jackut

POO / JAVA

Maceió/AL

2025

DAVI SILVA DE MELO LINS

Relatorio Milestone 3 - Projeto: Rede de Relacionamentos Jackut

POO / JAVA

Sistema Jackut - Rede Social

Apresentado no Instituto de Computação da
Universidade Federal de Alagoas, Campus A. C.
Simões, como parte dos requisitos para obtenção de
nota na disciplina de Programação 2 (P2) ou
Programação Orientada a Objetos (POO).

Orientador: Prof. Dr. Mario Hozano Lucas de Souza

Maceió/AL

2025

SUMÁRIO

1 OBJETIVOS

2 ESCOPO MILESTONE 3

3 METODOLOGIA

1.INTRODUÇÃO

Jackut é uma rede social inspirada em modelos clássicos de relacionamentos online, desenvolvida para demonstrar a aplicação de princípios de design de software e arquitetura modular. Este documento detalha a implementação das User Stories 10 a 13, correspondentes ao **terceiro** milestone do projeto, que abrange funcionalidades extras em relacionamentos, comunidades e outros.

2 Objetivos

A terceira etapa do projeto teve como objetivo principal a **evolução da implementação desenvolvida na Milestone 2**, incorporando as **críticas, sugestões e indicações recebidas durante a avaliação anterior**. Além disso, foram criadas novas User Stories 10 a 12, conforme as orientações passadas.

3 Escopo do Milestone 3

Na Milestone 3, foi pedido que fosse criado um novo módulo adicionando novas funcionalidades para a rede de relacionamento jackut, além da criação de pelo menos 20 casos de teste que satisfizesse as User Stories, além da implementação das mesmas no código.

O módulo que foi escolhido para ser implementado foi um módulo gerenciamento de comunidades.

3.1 Motivação do módulo de gerenciamento de comunidades.

Com o crescimento das comunidades, talvez seja necessário que algumas medidas de gerenciamento e moderação sejam tomadas, como expulsar usuários ou até mesmo bani-los. Sendo assim, será criado um módulo dedicado a isso.

3.2 User Stories:

- **US10** - Atribuição do cargo de moderação para usuários, além da listagem dos usuários com cargo de moderador;
- **US11** – Expulsão de membros das comunidades, com esses membros conseguindo entrar novamente na comunidade;
- **US12** – Banimento de membros das comunidades, além da listagem de usuários que estão banidos;
- **US13** – Desbanimento de membros que estão atualmente banidos da comunidade.

3.3 Testes para as user stories:

us10_1.txt:

US10.1 - Atribuir cargo de moderador a um membro da comunidade

zerarSistema

--- SETUP ---

Criação de usuários: dono, futuro moderador, membro comum e um usuário de fora

criarUsuario login=donoDaComunidade senha=123 nome="Dono"

sDono=abrirSessao login=donoDaComunidade senha=123

criarUsuario login=futuroMod senha=123 nome="Moderador"

sMod=abrirSessao login=futuroMod senha=123

criarUsuario login=membroComum senha=123 nome="Membro"

sMembro=abrirSessao login=membroComum senha=123

criarUsuario login=naoMembro senha=123 nome="Nao Membro"

sNaoMembro=abrirSessao login=naoMembro senha=123

Criação da comunidade e adição de membros

criarComunidade sessao=\${sDono} nome="Comunidade Top" descricao="Desc"

adicionarComunidade sessao=\${sMod} nome="Comunidade Top"

adicionarComunidade sessao=\${sMembro} nome="Comunidade Top"

--- TESTES DE LÓGICA E ERROS ---

1. Tenta atribuir moderador sendo um membro comum (deve falhar)

expectError "Apenas o dono ou moderadores podem realizar esta ação." atribuirModerador
sessao=\${sMembro} comunidade="Comunidade Top" login=membroComum

2. Tenta atribuir moderador a um usuário que não é membro (deve falhar)

```
expectError "Usuário não é membro da comunidade." atribuirModerador sessao=${sDono}  
comunidade="Comunidade Top" login=naoMembro
```

```
# 3. Dono atribui cargo de moderador com sucesso
```

```
atribuirModerador sessao=${sDono} comunidade="Comunidade Top" login=futuroMod
```

```
# 4. Verifica se o usuário agora é listado como moderador
```

```
expect {futuroMod} getModeradoresComunidade nome="Comunidade Top"
```

```
# 5. Tenta atribuir o cargo novamente para quem já é moderador (deve falhar)
```

```
expectError "Usuário já é moderador." atribuirModerador sessao=${sDono}  
comunidade="Comunidade Top" login=futuroMod
```

```
encerrarSistema
```

```
quit
```

us10_2.txt

US10.2 - Persistência e listagem de múltiplos moderadores

```
# --- TESTES DE PERSISTÊNCIA ---
```

```
# 1. Verifica se o cargo de moderador persistiu após reiniciar
```

```
expect {futuroMod} getModeradoresComunidade nome="Comunidade Top"
```

```
# Setup para novo teste
```

```
sDono=abrirSessao login=donoDaComunidade senha=123
```

```
sMembro=abrirSessao login=membroComum senha=123
```

```
# 2. Dono promove um segundo membro a moderador
```

```
atribuirModerador sessao=${sDono} comunidade="Comunidade Top" login=membroComum
```

```
# 3. Verifica a lista de moderadores com múltiplos usuários
```

```
expect {futuroMod,membroComum} getModeradoresComunidade nome="Comunidade Top"
```

```
# 4. Verifica se os membros originais (incluindo os promovidos) ainda constam na lista de membros
```

```
expect {donoDaComunidade,futuroMod,membroComum} getMembrosComunidade  
nome="Comunidade Top"
```

```
# 5. Verifica a persistência do dono da comunidade
```

```
expect donoDaComunidade getDonoComunidade nome="Comunidade Top"
```

```
encerrarSistema
```

```
quit
```

us11_1.txt

US11.1 - Expulsar um membro, que pode retornar em seguida

```
zerarSistema
```

--- SETUP ---

criarUsuario login=dono senha=123 nome="Dono"

sDono=abrirSessao login=dono senha=123

criarUsuario login=mod senha=123 nome="Mod"

sMod=abrirSessao login=mod senha=123

criarUsuario login=membroExpulso senha=123 nome="Expulso"

sMembroExpulso=abrirSessao login=membroExpulso senha=123

criarUsuario login=membroComum senha=123 nome="Membro"

sMembroComum=abrirSessao login=membroComum senha=123

criarComunidade sessao=\${sDono} nome="Comunidade" descricao="Desc"

adicionarComunidade sessao=\${sMod} nome="Comunidade"

adicionarComunidade sessao=\${sMembroExpulso} nome="Comunidade"

adicionarComunidade sessao=\${sMembroComum} nome="Comunidade"

atribuirModerador sessao=\${sDono} comunidade="Comunidade" login=mod

--- TESTES DE LÓGICA E ERROS ---

1. Um membro comum tenta expulsar outro (deve falhar)

expectError "Apenas o dono ou moderadores podem realizar esta ação." expulsarMembroComunidade
sessao=\${sMembroComum} comunidade="Comunidade" login=membroExpulso

2. Moderador expulsa um membro com sucesso

expulsarMembroComunidade sessao=\${sMod} comunidade="Comunidade" login=membroExpulso

3. Verifica se o usuário foi removido da lista de membros ativos

expect {dono,mod,membroComum} getMembrosComunidade nome="Comunidade"

4. Usuário que foi expulso consegue entrar na comunidade novamente

adicionarComunidade sessao=\${sMembroExpulso} nome="Comunidade"

5. Verifica se a lista de membros voltou ao estado original

expect {dono,mod,membroComum,membroExpulso} getMembrosComunidade nome="Comunidade"

encerrarSistema

quit

us11_2.txt

US11.2 - Persistência da expulsão com retorno

--- TESTES DE PERSISTÊNCIA ---

1. Verifica se a lista de membros (após o retorno do expulso) persistiu

expect {dono,mod,membroComum,membroExpulso} getMembrosComunidade nome="Comunidade"

Setup para novo teste

sDono=abrirSessao login=dono senha=123

sMod=abrirSessao login=mod senha=123

```
# 2. Moderador tenta expulsar o dono da comunidade (deve falhar)
expectError "Não é possível expulsar o dono da comunidade." expulsarMembroComunidade
sessao=${sMod} comunidade="Comunidade" login=dono

# 3. Dono agora expulsa o moderador
expulsarMembroComunidade sessao=${sDono} comunidade="Comunidade" login=mod

# 4. Verifica se o moderador foi removido da lista de membros
expect {dono,membroExpulso,membroComum} getMembrosComunidade nome="Comunidade"

# 5. Moderador expulso consegue retornar à comunidade
adicionarComunidade sessao=${sMod} nome="Comunidade"
expect {dono,membroExpulso,membroComum,mod} getMembrosComunidade nome="Comunidade"

encerrarSistema
quit
```

us12_1.txt

US12.1 - Banir um membro da comunidade

```
zerarSistema
```

```
# --- SETUP ---
```

```
criarUsuario login=dono senha=123 nome="Dono"
sDono=abrirSessao login=dono senha=123
criarUsuario login=mod senha=123 nome="Mod"
sMod=abrirSessao login=mod senha=123
criarUsuario login=membroParaBanir senha=123 nome="Banido"
sMembroBanido=abrirSessao login=membroParaBanir senha=123
criarUsuario login=membroComum senha=123 nome="Membro"
sMembroComum=abrirSessao login=membroComum senha=123

criarComunidade sessao=${sDono} nome="Comunidade Radical" descricao="Desc"
adicionarComunidade sessao=${sMod} nome="Comunidade Radical"
adicionarComunidade sessao=${sMembroBanido} nome="Comunidade Radical"
adicionarComunidade sessao=${sMembroComum} nome="Comunidade Radical"
atribuirModerador sessao=${sDono} comunidade="Comunidade Radical" login=mod
```

```
# --- TESTES DE LÓGICA E ERROS ---
```

```
# 1. Tenta banir a si mesmo (moderador) (deve falhar)
expectError "Não é possível banir a si mesmo." banirMembroComunidade sessao=${sMod}
comunidade="Comunidade Radical" login=mod
```

```
# 2. Dono bane um membro com sucesso
banirMembroComunidade sessao=${sDono} comunidade="Comunidade Radical"
login=membroParaBanir
```

3. Verifica se o membro banido está na lista de banidos
expect {membroParaBanir} getMembrosBanidosComunidade nome="Comunidade Radical"

4. Verifica se o membro foi removido da lista de membros
expect {dono,mod,membroComum} getMembrosComunidade nome="Comunidade Radical"

5. Moderador tenta banir o dono (deve falhar)
expectError "Não é possível banir o dono da comunidade." banirMembroComunidade
sessao=\${sMod} comunidade="Comunidade Radical" login=dono

encerrarSistema
quit

us12_2.txt

US12.2 - Persistência do banimento e múltiplos membros banidos

--- TESTES DE PERSISTÊNCIA ---

1. Verifica se a lista de banidos persistiu
expect {membroParaBanir} getMembrosBanidosComunidade nome="Comunidade Radical"

2. Verifica se a lista de membros ativos persistiu
expect {dono,mod,membroComum} getMembrosComunidade nome="Comunidade Radical"

Setup para novo teste
sMod=abrirSessao login=mod senha=123
sMembroBanido=abrirSessao login=membroParaBanir senha=123

3. Usuário banido tenta entrar na comunidade novamente (deve falhar)
expectError "Usuário está banido desta comunidade." adicionarComunidade
sessao=\${sMembroBanido} nome="Comunidade Radical"

4. Moderador bane outro membro
banirMembroComunidade sessao=\${sMod} comunidade="Comunidade Radical"
login=membroComum

5. Verifica a lista de banidos com múltiplos usuários
expect {membroParaBanir,membroComum} getMembrosBanidosComunidade nome="Comunidade Radical"

encerrarSistema
quit

us13_1.txt

US13.1 - Desbanir um membro da comunidade

zerarSistema

--- SETUP ---

```
criarUsuario login=dono senha=123 nome="Dono"
sDono=abrirSessao login=dono senha=123
criarUsuario login=mod senha=123 nome="Mod"
sMod=abrirSessao login=mod senha=123
criarUsuario login=membroBanido1 senha=123 nome="Banido1"
sMembroBanido1=abrirSessao login=membroBanido1 senha=123
criarUsuario login=membroBanido2 senha=123 nome="Banido2"
sMembroBanido2=abrirSessao login=membroBanido2 senha=123
```

```
criarComunidade sessao=${sDono} nome="Comunidade do Retorno" descricao="Desc"
adicionarComunidade sessao=${sMod} nome="Comunidade do Retorno"
adicionarComunidade sessao=${sMembroBanido1} nome="Comunidade do Retorno"
adicionarComunidade sessao=${sMembroBanido2} nome="Comunidade do Retorno"
atribuirModerador sessao=${sDono} comunidade="Comunidade do Retorno" login=mod
banirMembroComunidade sessao=${sDono} comunidade="Comunidade do Retorno"
login=membroBanido1
banirMembroComunidade sessao=${sMod} comunidade="Comunidade do Retorno"
login=membroBanido2
```

--- TESTES DE LÓGICA E ERROS ---

1. Valida o estado inicial com dois membros banidos

```
expect {membroBanido1,membroBanido2} getMembrosBanidosComunidade nome="Comunidade do Retorno"
```

2. Tenta desbanir um membro que não está banido (deve falhar)

```
expectError "Usuário não está banido desta comunidade." desbanirMembroComunidade
sessao=${sDono} comunidade="Comunidade do Retorno" login=mod
```

3. Dono desbane um membro com sucesso

```
desbanirMembroComunidade sessao=${sDono} comunidade="Comunidade do Retorno"
login=membroBanido1
```

4. Verifica se o membro foi removido da lista de banidos

```
expect {membroBanido2} getMembrosBanidosComunidade nome="Comunidade do Retorno"
```

5. Verifica que o membro desbanido ainda NÃO voltou a ser membro automaticamente

```
expect {dono,mod} getMembrosComunidade nome="Comunidade do Retorno"
```

```
encerrarSistema
```

```
quit
```

us13_2.txt

US13.1 - Desbanir um membro da comunidade

```
zerarSistema
```

--- SETUP ---

```
criarUsuario login=dono senha=123 nome="Dono"
sDono=abrirSessao login=dono senha=123
criarUsuario login=mod senha=123 nome="Mod"
sMod=abrirSessao login=mod senha=123
criarUsuario login=membroBanido1 senha=123 nome="Banido1"
sMembroBanido1=abrirSessao login=membroBanido1 senha=123
criarUsuario login=membroBanido2 senha=123 nome="Banido2"
sMembroBanido2=abrirSessao login=membroBanido2 senha=123
```

```
criarComunidade sessao=${sDono} nome="Comunidade do Retorno" descricao="Desc"
adicionarComunidade sessao=${sMod} nome="Comunidade do Retorno"
adicionarComunidade sessao=${sMembroBanido1} nome="Comunidade do Retorno"
adicionarComunidade sessao=${sMembroBanido2} nome="Comunidade do Retorno"
atribuirModerador sessao=${sDono} comunidade="Comunidade do Retorno" login=mod
banirMembroComunidade sessao=${sDono} comunidade="Comunidade do Retorno"
login=membroBanido1
banirMembroComunidade sessao=${sMod} comunidade="Comunidade do Retorno"
login=membroBanido2
```

--- TESTES DE LÓGICA E ERROS ---

1. Valida o estado inicial com dois membros banidos

```
expect {membroBanido1,membroBanido2} getMembrosBanidosComunidade nome="Comunidade do Retorno"
```

2. Tenta desbanir um membro que não está banido (deve falhar)

```
expectError "Usuário não está banido desta comunidade." desbanirMembroComunidade
sessao=${sDono} comunidade="Comunidade do Retorno" login=mod
```

3. Dono desbane um membro com sucesso

```
desbanirMembroComunidade sessao=${sDono} comunidade="Comunidade do Retorno"
login=membroBanido1
```

4. Verifica se o membro foi removido da lista de banidos

```
expect {membroBanido2} getMembrosBanidosComunidade nome="Comunidade do Retorno"
```

5. Verifica que o membro desbanido ainda NÃO voltou a ser membro automaticamente

```
expect {dono,mod} getMembrosComunidade nome="Comunidade do Retorno"
```

encerrarSistema

quit

CONCLUSÃO:

Os testes e as user stories foram criadas e implementados no código com sucesso, falhando apenas em alguns casos em que o comportamento que obtive foi a lista sendo devolvida na ordem incorreta, tal bug tendo ocorrido a partir do momento que transformei a entidade user em SRP, sendo os erros análogos aos que obtive na milestone 2, então haverá mais detalhes no relatório da mesma. Em relação a milestone 2, basicamente foi melhorada a parte de SRP e design patterns, na qual eu resolvi seguir com uma **FACADE** para a parte dos serviços, o qual me possibilitou me adequar ao SRP e utilizar um design pattern de fato no meu código, algo que eu não fizera na entrega da Milestone 2, tendo mais detalhes do approach que eu tomei no relatório da mesma.